

Centralized Return, Refund, Dispute & Platform Payment Resolution

Senior Full-Stack Architecture, Routing, Database, Workflow & Verification Plan

This specification refactors the existing seller-controlled return/refund workflow to align with LumBarong's centralized GCash/Maya payment model and cash-only seller commission model.

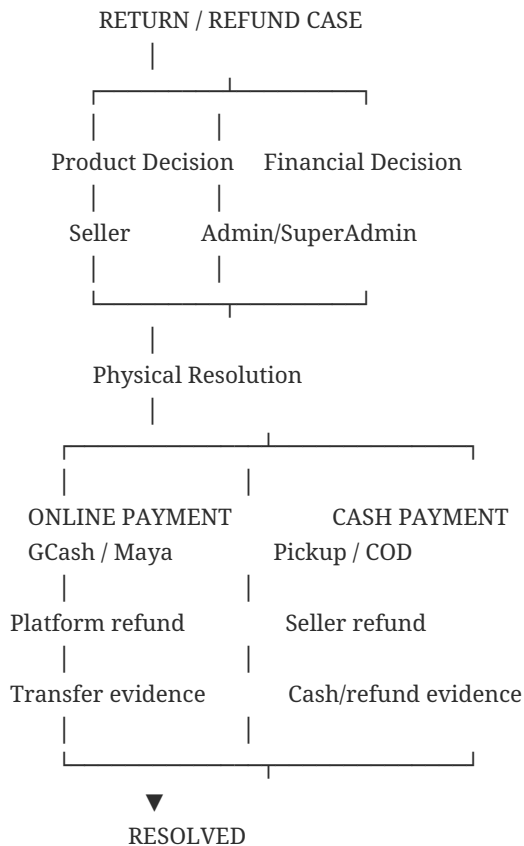
Payment Class	Who Receives the Money?	Who Handles Refund?
GCash / Maya	LumBarong platform account	Authorized Admin / SuperAdmin
Store Pickup + Cash	Seller / artisan	Seller / artisan
Special Delivery + COD	Seller / rider	Seller / artisan, with platform record

KEY ARCHITECTURAL RULE

Return approval $\neq$ physical return completion $\neq$ refund transfer

1. Executive Architecture Decision

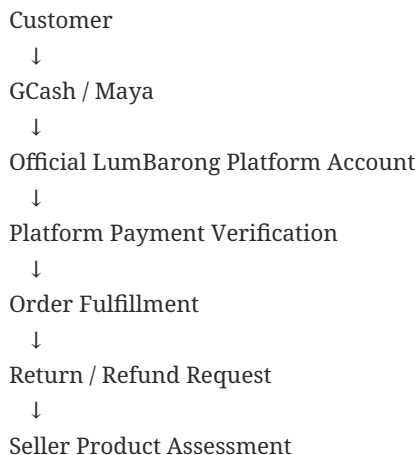
The new model should separate four business concerns: product/return eligibility, physical item movement, financial refund processing, and dispute resolution. A seller can assess a product issue without being given authority to transfer money that the platform collected.



Use terms such as platform-held payment or platform payment balance in code and UI unless the actual payment arrangement legally constitutes escrow. The system should not label funds as “escrow” merely because the platform receives them.

2. Core Business Model

2.1 Online GCash / Maya



↓
Admin / SuperAdmin Resolution
↓
Platform Refund

2.2 Cash: Store Pickup

Customer
↓
Store Pickup
↓
Seller / Artisan collects cash
↓
Return / Exchange request
↓
Seller cash resolution
↓
Platform records result

2.3 Cash: Special Delivery / COD

Customer
↓
Special Delivery / COD
↓
Seller / Rider collects cash
↓
Return / Refund request
↓
Seller cash resolution
↓
Platform records result

3. State Model

Do not use one generic status such as refunded to represent the entire lifecycle. Separate return status, physical return status, and refund status. A request may be approved while the physical item is still in transit and the refund is still pending.

Domain	Recommended States
Return Request	submitted, seller_review, approved, rejected, awaiting_return, return_in_transit, return_received, admin_review, disputed, resolved, cancelled
Refund	not_applicable, pending, approved, processing, transferred, failed, rejected
Physical Return	not_required, awaiting_customer, in_transit, received, inspected, completed
Dispute	none, opened, under_review, resolved

Return: APPROVED

Physical: AWAITING_RETURN

Refund: PENDING

4. Mandatory Pre-Implementation Audit

Before changing code, trace the actual implementation rather than assuming the existing `RefundController.php` matches documentation.

Area	Inspect
Controllers	app/Http/Controllers/RefundController.php; CheckoutController.php; AdminController.php; SuperAdminController.php
Models	Order; OrderItem; Product; User; PaymentTransaction; CommissionRecord; existing Return/Refund models
Views	resources/views/orders; seller/returns; admin/returns; superadmin/returns; related order pages
Routes	routes/web.php; routes/api.php; current return/refund endpoints and middleware
Database	return/refund tables; payment tables; commission records; indexes; foreign keys; status fields
Frontend	evidence upload; modals; AJAX/fetch; state display; receipt/video viewers; notifications

Search the complete codebase for: refund, return, returnRequest, refundRequest, approve, reject, paymentStatus, paymentMethod, GCash, Maya, COD, Store Pickup, Special Delivery.

5. Responsibility Matrix

Actor	May Do	Must Not Do
Customer	Submit return; upload evidence; dispute; provide/confirm refund destination; view status	Set authoritative refund amount; approve own refund; access another customer case
Seller / Artisan	Inspect evidence; recommend approve/reject; confirm returned item; perform cash refund; arrange replacement/exchange	Transfer platform GCash/Maya refunds; alter platform refund amount; approve platform financial transfer
Admin	Review case; request evidence; approve/reject refund; process platform refund; upload transfer proof; mediate disputes	Bypass authorization; over-refund; modify unrelated cases
SuperAdmin	All admin actions plus authorized overrides and platform-wide audit	Bypass business validation or create untraceable financial state

6. Customer Return Entry & Eligibility

Recommended customer entry points, subject to the existing route conventions:

```

GET /orders/{order}/returns/create
POST /orders/{order}/returns
GET /orders/{order}/returns/{returnRequest}
POST /orders/{order}/returns/{returnRequest}/dispute
POST /orders/{order}/returns/{returnRequest}/evidence

```

Before accepting a request, the server must confirm order ownership, item ownership, receipt/claim status, return-window eligibility, and that the case is not already finalized.

Return reasons should support: Damaged Item, Wrong Item, Wrong Size / Variation, and Other. Customer explanations and photo/video proof should be persisted as case evidence.

6.1 Evidence Security

```

return_refund_evidences
- id
- return_request_id
- uploaded_by
- type
- storage_path
- mime_type
- file_size
- checksum
- created_at

```

Use server-side MIME validation, size limits, safe/private storage where appropriate, controlled access, and protections against executable uploads, path traversal, and unintended public exposure.

7. Seller / Artisan Review Workflow

```

GET /seller/returns
GET /seller/returns/{returnRequest}
POST /seller/returns/{returnRequest}/review
POST /seller/returns/{returnRequest}/receive
POST /seller/returns/{returnRequest}/cash-refund
POST /seller/returns/{returnRequest}/replacement

```

Seller review represents the product-side assessment. Approval by the seller must not itself mark an online refund as financially transferred.

```

Seller assessment: APPROVED
Next stage: Admin Refund / Resolution Review

```

8. Admin & SuperAdmin Resolution Hub

```

GET /admin/returns
GET /admin/returns/{returnRequest}
POST /admin/returns/{returnRequest}/resolve
POST /admin/returns/{returnRequest}/request-evidence
POST /admin/returns/{returnRequest}/approve-refund
POST /admin/returns/{returnRequest}/record-transfer
POST /admin/returns/{returnRequest}/reject-refund
POST /admin/returns/{returnRequest}/resolve-dispute

```

GET /superadmin/returns

GET /superadmin/returns/{returnRequest}

POST /superadmin/returns/{returnRequest}/resolve

POST /superadmin/returns/{returnRequest}/approve-refund

POST /superadmin/returns/{returnRequest}/record-transfer

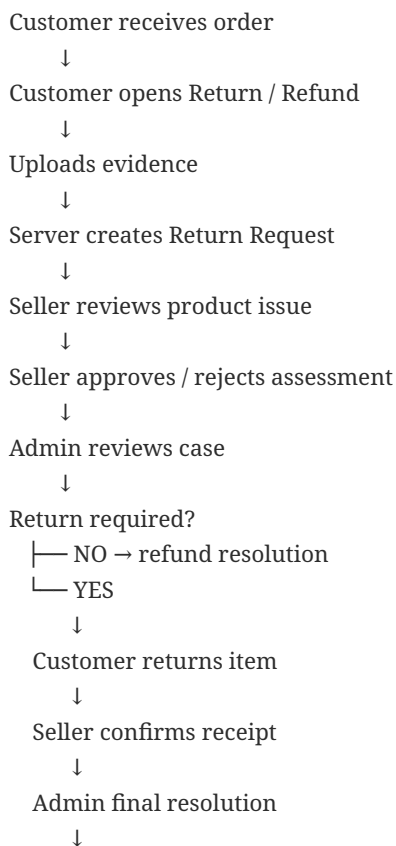
POST /superadmin/returns/{returnRequest}/override

Use the project's current route naming and middleware if equivalent routes already exist. Do not create duplicate endpoints unnecessarily.

9. Admin Return / Refund Detail Screen

Section	Required Data
Customer	Name, email/contact as authorized, order ID
Seller	Seller/shop
Order	Order status, fulfillment method, payment method, dates
Return	Reason, explanation, affected items, requested amount, seller decision/notes, admin decision/notes
Evidence	Photos, videos, supporting files, viewer/lightbox
Financial	Original paid amount, eligible refund, approved refund, refund destination, refund status

10. Online GCash / Maya Return & Refund Flow



Refund approved

↓

Authorized admin transfers funds

↓

Transfer proof/reference recorded

↓

Refund = TRANSFERRED

↓

Case = RESOLVED

Do not set refund_status = completed at the moment of approval. Financial completion occurs only after the actual transfer is confirmed and evidence/reference is recorded.

11. Online Refund Destination & Security

refund_method
refund_account_name
refund_account_number

Prefer the same provider as the original verified payment unless the platform has an explicit policy for another destination. Mask account numbers in list views, encrypt sensitive destination data where appropriate, and restrict full visibility to authorized refund processors.

09*****1234

12. Refund Transaction & Amount Controls

refund_transactions
- id
- return_request_id
- order_id
- payment_method
- refund_method
- refund_amount
- destination_account
- destination_account_masked
- status
- transfer_reference
- transfer_proof_path
- processed_by
- processed_at
- failure_reason
- timestamps

Reuse existing payment/refund tables when they already provide equivalent normalized structures. Avoid unnecessary parallel models.

Refund amounts must be server-authoritative. The maximum refundable amount should be derived from the captured/verified payment and previous refunds.

Paid amount = P1,500
Already refunded = P500
Maximum remaining refund = P1,000

Database transactions, row locking where appropriate, unique constraints, and idempotency checks must prevent over-refunding and duplicate refund transactions.

13. Partial Refunds & No-Return Resolutions

Support full, partial, and item-level refunds where the current business policy allows. Example: a P3,000 order contains a P1,000 defective item; the server should calculate a P1,000 maximum for that item rather than defaulting to the full order total.

Not every case requires a physical return. Where policy permits, an authorized admin may select `refund_without_return`. Otherwise track the physical return lifecycle explicitly.

14. Cash Return / Refund Flows

14.1 Store Pickup + Pay in Shop

```

graph TD
    A[Customer claims order] --> B[Discovers wrong size / defect]
    B --> C[Return or exchange case]
    C --> D[Seller inspects]
    D --> E[Exchange / adjustment / cash refund]
    E --> F[Seller records resolution]
    F --> G[Customer sees resolved status]
  
```

14.2 Special Delivery + COD

```

graph TD
    A[Customer pays rider / seller in cash] --> B[Problem discovered]
    B --> C[Return request]
    C --> D[Seller inspects]
    D --> E[Item returned / adjusted]
    E --> F[Seller gives cash refund if applicable]
    F --> G[Platform records resolution]
  
```

Even when the seller controls the cash, the platform should record who performed the refund, the amount, the resolution type, and the date for auditability.

15. Dispute Resolution

Seller rejects
 ↓
 Customer sees reason
 ↓
 Customer disputes
 ↓
 Admin review
 ↓
 Request more evidence if needed
 ↓
 Admin decision
 ↓
 Approve / Reject / Request Evidence
 POST /orders/{order}/returns/{returnRequest}/dispute
 POST /admin/returns/{returnRequest}/resolve-dispute
 POST /superadmin/returns/{returnRequest}/override

A dispute is a continuation of the same return case, not an unrelated new order request.

16. Customer Timeline & Notifications

- ✓ Return request submitted
- ✓ Seller reviewed request
- ✓ Return approved
- ✓ Item received by seller
- ✓ Refund approved
- Refund being transferred
- Refund completed

Notify customers when requests are submitted, approved/rejected, evidence is requested, a returned item is received, a refund is approved, the transfer succeeds/fails, or a dispute is resolved. Keep internal admin notes private.

17. Commission Interaction

The return/refund model must integrate with the cash-only commission model.

Transaction	Initial Commission	Refund Impact
GCash / Maya	No seller cash debt merely because the order exists	Platform-side refund/reconciliation; do not create seller cash debt
Store Pickup + Cash	Commission applies when seller-collected cash transaction reaches the defined commission event	If fully refunded, create a traceable reversal/adjustment according to policy
Special Delivery + COD	Commission applies when seller-collected cash transaction reaches the defined commission event	Refund should produce a financial adjustment rather than deleting history

Original commission: P100

Refund adjustment: -P100

Net commission: P0

Inspect the existing CommissionRecord lifecycle before changing it. Preserve finalized financial history and prefer ledger-style adjustments/reversals over destructive deletion.

18. Cancellation vs. Return

Cancellation before fulfillment

↓

Existing cancellation workflow

↓

Refund infrastructure reused if payment was captured

Delivered / claimed order

↓

Return request

↓

Product assessment

↓

Refund / exchange / adjustment

Do not classify an order the customer never received as a normal return.

19. Recommended Laravel Architecture

app/Http/Controllers/

Customer/ReturnRequestController.php

Seller/ReturnReviewController.php

Admin/ReturnManagementController.php

Admin/RefundManagementController.php

SuperAdmin/ReturnManagementController.php

SuperAdmin/RefundManagementController.php

app/Services>Returns/

CreateReturnRequestService.php

EvaluateReturnEligibilityService.php

SellerReturnReviewService.php

ResolveReturnRequestService.php

ProcessRefundService.php

RecordCashRefundService.php

DisputeReturnRequestService.php

CommissionRefundAdjustmentService.php

Use the smallest clean architecture that fits the actual project. Do not split controllers/services solely for appearance if the existing application has a proven organization pattern.

20. Routing & Authorization Matrix

Actor	Route	Authorization	Purpose
Customer	POST /orders/{order}/returns	auth + customer/order policy	Create return

Actor	Route	Authorization	Purpose
Customer	POST /orders/{order}/returns/{returnRequest}/dispute	auth + ownership policy	Open dispute
Seller	POST /seller/returns/{returnRequest}/review	auth + seller + policy	Product assessment
Seller	POST /seller/returns/{returnRequest}/cash-refund	auth + seller + policy	Record cash refund
Admin	POST /admin/returns/{returnRequest}/approve-refund	auth + admin + refund permission	Approve platform refund
Admin	POST /admin/returns/{returnRequest}/record-transfer	auth + admin + refund permission	Record actual transfer
Admin	POST /admin/returns/{returnRequest}/resolve-dispute	auth + admin + dispute permission	Resolve dispute
SuperAdmin	POST /superadmin/returns/{returnRequest}/override	auth + superadmin	Authorized override

21. Security & Concurrency Requirements

- Prevent IDOR and cross-seller/cross-customer access using policies and server-side authorization.
- Prevent refund amount manipulation; never trust browser-supplied financial totals.
- Protect evidence files and refund destinations with validated uploads and controlled access.
- Make approval and transfer transitions idempotent and safe against double-clicks, retries, and concurrent admin actions.
- Require audit logging for financial and override actions.

Two admins approve simultaneously

↓

One valid state transition

↓

Second request returns already processed / conflict

22. Situational Scenarios

Scenario 1 - GCash + Damaged Item + Successful Refund

Order P1,800; GCash; customer uploads unboxing video/photos; seller validates defect; customer returns item; seller confirms receipt; admin approves P1,800; authorized staff transfers to customer GCash; transfer proof/reference recorded; case resolved.

Scenario 2 - Maya + Wrong Size + Replacement

Customer received size L instead of M; seller confirms error; customer returns item; seller receives it; replacement is issued. Refund status remains not_applicable; resolution is replacement.

Scenario 3 - Store Pickup + Cash + Immediate Exchange

Customer pays cash at shop, discovers wrong size while/after claiming, seller exchanges for correct size. Platform records exchange; no online refund transaction is created.

Scenario 4 - Store Pickup + Cash Refund

Customer returns a damaged item; seller returns P1,200 cash; server verifies the amount against the original order/payment; seller records the completed cash refund; customer sees resolved status.

Scenario 5 - COD + Damaged Item

Customer paid P2,000 cash for Special Delivery; seller accepts return and refunds P2,000 cash; platform records cash refund and applies the defined commission adjustment/reversal.

Scenario 6 - Seller Rejects, Customer Disputes

Seller rejects; customer disputes; admin reviews evidence, may request more proof, and makes the final resolution. The seller cannot block escalation.

Scenario 7 - Duplicate Refund Attempt

Admin submits the same refund twice; first succeeds; second is rejected as already processed. Database/service idempotency prevents double payment.

Scenario 8 - Wrong Refund Destination

Original payment provider is Maya; customer enters a different destination. Admin must validate/confirm destination according to policy before transfer; full sensitive account data is restricted.

Scenario 9 - Refund Transfer Fails

Transfer attempt fails; refund status becomes failed with a reason. It must not become transferred; retry occurs through a controlled flow without creating a duplicate refund.

Scenario 10 - Cancellation Before Shipment

Customer paid GCash but order was never received. Existing cancellation workflow handles the refund; a normal post-delivery return request is not created.

Scenario 11 - Partial Refund

Order P3,000; only a P1,000 item is defective. Refund is capped at the eligible P1,000 amount unless policy explicitly includes other components.

Scenario 12 - Payment Not Yet Verified

Customer submits a return on an order with unverified GCash/Maya payment. The system does not process a normal completed-payment refund until the payment state is resolved.

23. Recommended UI Structure

Role	Recommended Sections
Customer	My Orders > Return/Refund; evidence upload; case timeline; dispute; refund destination; refund status
Seller	Returns; pending review; returned items; cash refund/effect resolution; replacement/exchange
Admin	Return queue; pending seller review; admin review; awaiting return; pending refund; refund processing; disputes; completed
SuperAdmin	All admin views plus overrides, platform-wide audit, financial reconciliation

24. Database Design Checklist

return_requests
 - order_id
 - order_item_id (nullable if order-level)
 - customer_id
 - seller_id
 - reason
 - customer_message
 - return_status
 - physical_return_status
 - requested_amount
 - approved_amount
 - seller_decision
 - seller_notes
 - admin_decision
 - admin_notes
 - dispute_status
 - resolved_by
 - resolved_at
 - timestamps

Add fields only when equivalent existing columns are absent. Preserve historical orders and finalized financial records. Use foreign keys and indexes appropriately.

25. Testing Strategy

Test Group	Required Coverage
Customer	Eligibility; ownership; evidence uploads; dispute; additional evidence; timeline
Seller	Own-case access; product review; receive return; cash refund; replacement; amount limits; no platform refund ability
Admin	Review; evidence request; refund approval; rejection; transfer record; over-refund protection; dispute resolution

Test Group	Required Coverage
SuperAdmin	Override; all-case visibility; auditability
Payment	GCash/Maya verified flow; duplicate reference; unverified payment handling; refund amount cap; transfer failure/retry
Cash	Store Pickup cash refund/exchange; COD refund; commission reconciliation
Concurrency	Double-click; two admins; retry after timeout; duplicate worker processing
Regression	Checkout; payment verification/OCR; shipping; Store Pickup; Special Delivery; seller orders; commissions; dashboards

Do not report a feature as verified unless the relevant tests actually pass. Run the complete existing test suite after the affected feature tests.

26. Final Acceptance Criteria

- Customer can submit an eligible return request with reason and evidence.
- Seller can inspect product issues but cannot control platform online refunds.
- Admin/SuperAdmin can resolve centralized GCash/Maya refunds.
- Cash refunds remain seller-controlled and are still recorded by the platform.
- Return, physical-return, refund, and dispute states are distinct.
- Refund amounts are server-authoritative and cannot exceed the remaining refundable amount.
- Duplicate refund processing is prevented with transactions/idempotency/constraints.
- Refund transfer requires appropriate confirmation evidence or reference.
- Customer can view the refund lifecycle.
- Seller decisions can be disputed and escalated to admin.
- SuperAdmin overrides are controlled and audited.
- Commission records remain consistent after full/partial refunds.
- Cancellation-before-delivery does not become a normal return case.
- Unauthorized users cannot access or mutate another party's case.
- Existing payment-proof/OCR, shipping, Store Pickup, Special Delivery, checkout, and order workflows continue to pass regression tests.

27. Required Implementation Output

After implementation, the coding agent must produce a structured report containing:

- Existing Architecture Discovered
- Problems Found (Critical / Major / Moderate / Minor)
- Architecture Changes
- Routes Added/Modified
- Controllers and Services
- Database Changes
- UI Changes

- Online Payment Flow
- Cash Flow
- Commission Reconciliation
- Security Controls
- Actual Test Results
- Remaining Issues

Tests run: <actual value>

Passed: <actual value>

Failed: <actual value>

Assertions: <actual value>

Never claim VERIFIED, COMPLETE, or PRODUCTION-READY without evidence from actual implementation checks and tests.

28. Final Engineering Rule

Treat this feature as a financial workflow redesign, not a simple Refund button. Every implementation decision must answer: who received the money, who validates the product problem, who authorizes the refund, who physically transfers/refunds the money, how the result is recorded, what happens to commission, what happens when transfer fails, and what happens when the customer disputes the seller.

Payment

↓

Order

↓

Return Case

↓

Product Assessment

↓

Physical Resolution

↓

Financial Resolution

↓

Commission Reconciliation

↓

Audit + Notification

↓

Resolved

End of Implementation Specification